# SENTINEL Submission Guide

*The marks, the oral exams and the hand-in of each lab*

ECE 476 Engineering System Software • Fall 2026 • University of Victoria

## 1 The marks of each lab

Each lab gives a part of the course grade. The four labs together give 26%.

| Lab | Session | Deliverable due | Deliverable | Oral exam | Oral exam date | Lab |
|---|---|---|---|---|---|---|
| 1 | Oct 9 | Oct 22, by midnight | 4% | 2% | Oct 30, in the Lab 2 session | 6% |
| 2 | Oct 30 | Nov 12, by midnight | 4% | 2% | Nov 20, in the Lab 3 session | 6% |
| 3 | Nov 20 | Dec 2, by midnight | 4% | 2% | Dec 4, in the Lab 4 session | 6% |
| 4 | Dec 4 | Dec 11 | 4% | 4% | no oral exam: a video, due Dec 11 | 8% |
| **All four labs** | | | | | | **26%** |

Each lab also has a prelab,[1] which is due before the session of the lab. The brief of each lab[2] states the date of its prelab. It also states how the marker divides the marks of the deliverable and of the oral exam.

## 2 The deliverable

The deliverable of a lab is one zip file, which you upload to Brightspace before the deadline. `make submit` makes the zip (Section 5). You do not hand in your repository: you do not tag it or push it for the marker, and the marker does not read it.

The brief of each lab lists the files that the lab adds to the zip. For Lab 4, you upload the zip and the video together, by Dec 11.

## 3 The oral exam, Labs 1 to 3

The oral exam of a lab is in the session of the next lab. The table in Section 1 gives each date.

The oral exam takes five minutes, at your own machine, with your code on the screen. First you show your lab: you run it, and you show that it works. Then the examiner asks questions about your code and about the theory of the lab. The brief of each lab states the questions of its oral exam, and the prelab is one source of them.

The oral exam tests that you understand the code that you submitted. To describe what a program does is one skill. To say what it will do in a condition that you did not try is a different skill. Only the second skill needs a model of the operating system.

[1] The Lab 1 prelab, `docs/lab1/prelab1.pdf` in your repository. It is due before the Lab 1 session.
[2] The Lab 1 Brief, `docs/lab1/lab1_brief.pdf` in your repository. It states the contract of Lab 1.

To prepare, build the lab yourself, and record each fault that you find in `docs/DEFECTS.md`.[3]

## 4 The video, Lab 4

Lab 4 has no oral exam. A video replaces it, and it gives 4% of the course grade. In the video, you show the whole system from end to end, you talk through the project, and you explain how each part works.

The video is due on Dec 11, with the Lab 4 deliverable. The Lab 4 brief states the length of the video, its file format, and where you upload it.

## 5 Build the zip

1. Make the files that the brief of your lab lists for the hand-in. For Lab 1, these are the three files of one run in `docs/lab1/`.[4] For Lab 2, these are the three files in `docs/lab2/`.[5]
2. At the top of your repository, run one of these commands:

```
make submit                       # Lab 1; the zip has your login name
make submit NAME=<your name>      # the zip has the name that you give
make submit LAB=2                 # the zip of Lab 2
```

`make submit` does these steps in this order:

1. It checks that your computer has `zip`.
2. It removes `build/`, and it builds each part again from the source.
3. It runs your tests, as `make test`[6] does.
4. It runs the acceptance test of the lab: `make accept1`,[7] or `make accept2`.[8]
5. It checks that each file of the lab exists and is not empty.
6. It writes the zip, and it prints the name, the number of files and the size of the zip. If a test or an acceptance check failed, it says so next to the name.

Only two faults stop `make submit`, and then it writes no zip: a build that fails (step 2), and a file of the lab that does not exist or is empty (step 5). A test or a check that fails does not stop it, so you can hand in work that is not complete. The brief of each lab states how the checks give marks.

[3] `docs/DEFECTS.md` is your record of each fault that you find: what you saw, the cause, and the fix. The SENTINEL Lab Manual, Section 6, `docs/manual/sentinel_manual.pdf`, describes it.

[4] The three files of one run of the Lab 1 demo. The Lab 1 Brief, Section 6.2, `docs/lab1/lab1_brief.pdf`, gives the commands that make them.

[5] The three files of the hand-in of Lab 2. The Lab 2 Brief, Section 6, `docs/lab2/lab2_brief.pdf`, gives the commands that make them.

[6] `make test` runs the tests of each lab that has a `Makefile`. The SENTINEL Lab Manual, Section 4, `docs/manual/sentinel_manual.pdf`, lists each make target.

[7] The acceptance test of Lab 1, `make accept1`, runs 34 checks of your library and your demo. The Lab 1 Brief, Section 4.3, `docs/lab1/lab1_brief.pdf`, describes it.

[8] The acceptance test of Lab 2, `make accept2`, runs 26 checks of your library and of the demo. The Lab 2 Brief, Section 5, `docs/lab2/lab2_brief.pdf`, describes it.

**It builds from the source for a reason.** A source file that your `Makefile` does not name can build on your machine. Its object file is still in `build/` from an earlier run. That file does not build on the machine of the marker. The build from the source finds this fault before you upload.

The zip is `sentinel-lab1-<name>.zip` for Lab 1, and `sentinel-lab2-<name>.zip` for Lab 2, at the top of your repository. It holds `Makefile`, `README.md`, `.gitignore`, `common/`, `docs/`, `src/`, `tools/` and `web/`. It does not hold `build/`, `.git/`, `.vscode/`, `web/live`, a library (`*.a`), a dependency file (`*.d`), a core file (`core`, `vgcore.*`) or a zip. The marker builds your code again from the source in the zip.

## 6 Upload the zip

Upload the zip to Brightspace before the deadline of the lab. Keep the given files as the course sent them. The marker compares your files with the files that the course released, and you must explain each given file that you changed.

## 7 Faults, and what each one means

| What you see | What it means, and what to do |
|---|---|
| `zip is not installed` | Install it: `sudo apt install zip` |
| `docs/lab1/<file>` or `docs/lab2/<file> is missing or empty` | Make the files of the hand-in. For Lab 1, the three files of one run;[9] for Lab 2, the three files of its hand-in.[10] |
| The build stops, but `make` worked before | Your `Makefile` does not name a source file. Your earlier builds used an old object file from `build/`, which `make submit` removed |
| `A test failed`, or `Some acceptance checks failed`, next to the name of the zip | The zip exists, and it holds your work as it is now. If you have time, find the fault, and run `make submit` again |

[9] The three files of one run of the Lab 1 demo. The Lab 1 Brief, Section 6.2, `docs/lab1/lab1_brief.pdf`, gives the commands that make them.

[10] The three files of the hand-in of Lab 2. The Lab 2 Brief, Section 6, `docs/lab2/lab2_brief.pdf`, gives the commands that make them.